



Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura

Programación web de lado del servidor

Tema

Tema 2 API's

EVIDENCIA DE APRENDIZAJE

AA 2.1 API Reporte practico

Profesor

MTI. Paloma Gongora Sabido

Alumno (s):

Jairo Amir CahunTamay

(Semestre y Grupo) ISC-8B

Felipe Carrillo Puerto a 22 de marzo de 2026



2026
año de
**Margarita
Maza**





1. Interfaz de Programación de Aplicaciones

Las API, es la tecnología que potencia la comunicación entre el software en Internet. API son las siglas de Application Programming Interface (interfaz de programación de aplicaciones), Es un conjunto de normas y protocolos que definen cómo pueden interactuar distintos programas informáticos. Si el mundo entero funciona con software, es crucial pensar en la comunicación entre los diferentes programas que existen.

Ejemplo:

A partir de dos programas, diferentes Programa A (habla maya) y Programa B (habla inglés). Para que ambos interactúen, necesitan una interfaz que sirva de puente entre ellos.

Tipos de APIs

Existen diferentes tipos de API, como GraphQL, SOAP, REST, gRPC, etc. Si se compara la creación de las APIs con la arquitectura de edificios; cada uno elige su estilo.

La API REST es la más importante para el desarrollo web. Una API RESTful sigue un conjunto específico de reglas, utilizando el protocolo HTTP para interactuar.

Petición a API

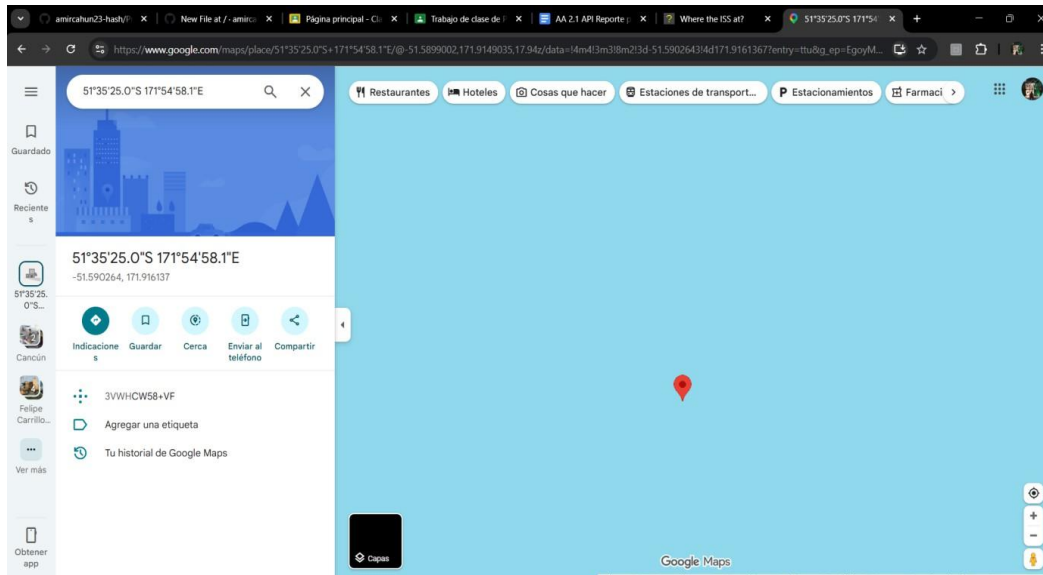
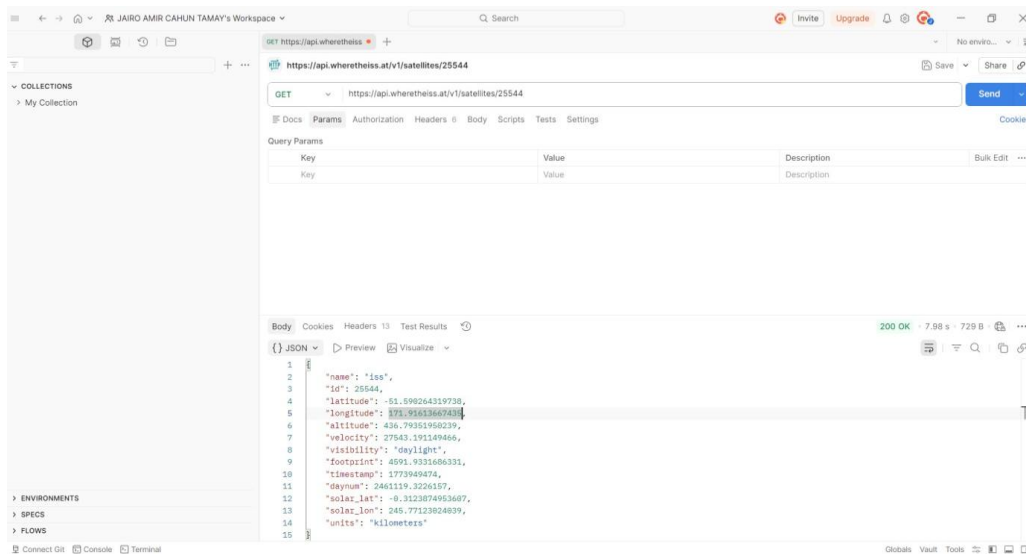
La API REST de WTIA, es un servicio, para obtener la posición actual, pasada o futura de la ISS, información de la zona horaria de un conjunto de coordenadas y datos TLE de la ISS.

La Estación Espacial Internacional (ISS) es un satélite artificial habitable en órbita baja terrestre. Es un proyecto conjunto de cinco agencias espaciales participantes: la NASA estadounidense, la RSA rusa, la JAXA japonesa, la ESA europea y la CSA canadiense. La tripulación actual, compuesta por seis personas, forma parte de la Expedición 35, comandada por Chris Hadfield, de la CSA.

1. Solicitud GET para obtener latitud y longitud de la ISS.

- Usar Postman para hacer la solicitud HTTP (sin frontend o backend).
- Utilizar los valores latitud y longitud obtenidos para buscar la ubicación de la ISS en Google Maps.



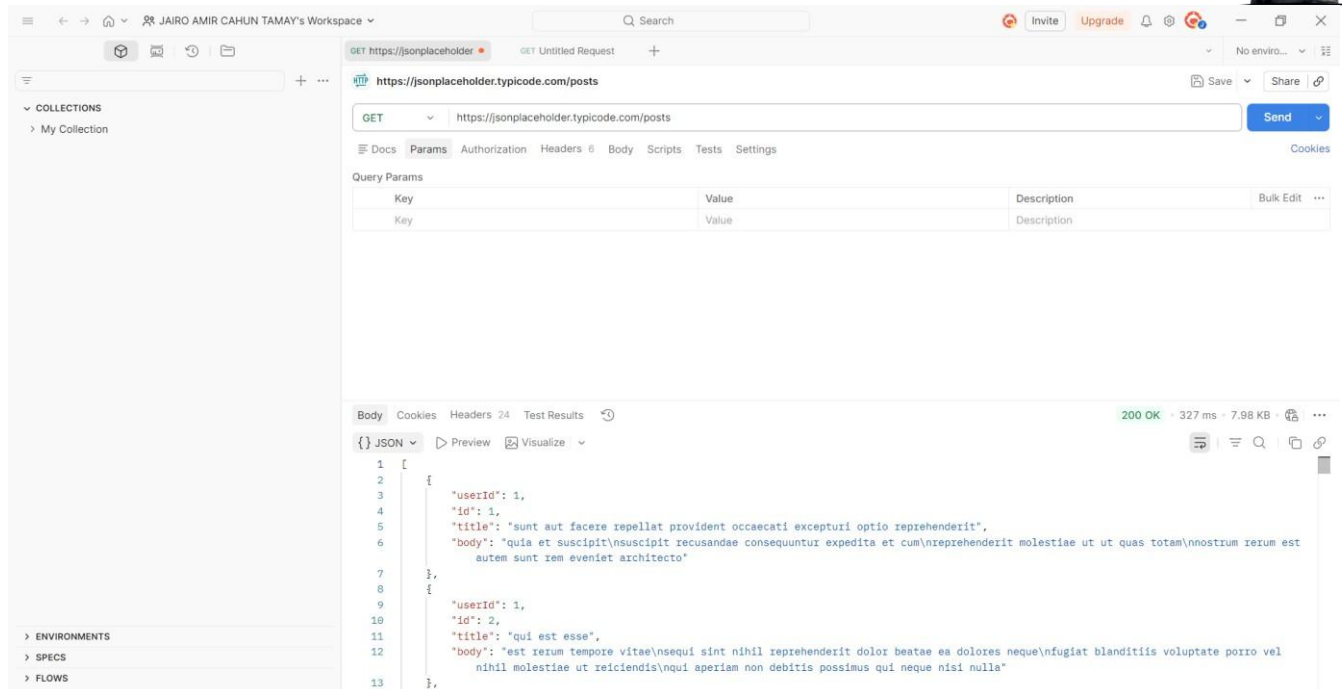


Petición a API

JSON Placeholder es un servicio gratuito que proporciona datos ficticios para pruebas y prototipos. Puedes usarlo para obtener información sobre usuarios, publicaciones, comentarios, etc.

1. Utiliza Postman para realizar una solicitud GET a la siguiente URL para obtener una lista de usuarios: <https://jsonplaceholder.typicode.com/users>
2. La API devolverá un objeto JSON que contiene una lista de usuarios.



GET https://jsonplaceholder.typicode.com/posts

200 OK · 327 ms · 7.98 KB

```
{
  "posts": [
    {
      "userId": 1,
      "id": 1,
      "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
    },
    {
      "userId": 1,
      "id": 2,
      "title": "qui est esse",
      "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
    }
  ]
}
```

3. Puedes mostrar la información de los usuarios en tu aplicación o sitio web. Por ejemplo, podrías listar los nombres y correos electrónicos de los usuarios.

4. Para obtener otro tipo de recurso, puedes hacer una solicitud GET a:

Resources

JSONPlaceholder comes with a set of 6 common resources:

/posts	100 posts
/comments	500 comments
/albums	100 albums
/photos	5000 photos
/todos	200 todos
/users	10 users





1.

GET https://jsonplaceholder.typicode.com/comments/1

GET https://jsonplaceholder.typicode.com/comments/1

Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "postId": 1,
  "id": 1,
  "name": "id labore ex et quam laborum",
  "email": "Eliseo@gardner.biz",
  "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
}
```

2.

GET https://jsonplaceholder.typicode.com/comments/1

GET https://jsonplaceholder.typicode.com/comments/1

Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

```
{
  "postId": 1,
  "id": 1,
  "name": "id labore ex et quam laborum",
  "email": "Eliseo@gardner.biz",
  "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
}
```





3.

JAIRO AMIR CAHUN TAMAY's Workspace

GET https://jsonplaceholder.typicode.com/photos/1

GET https://jsonplaceholder.typicode.com/photos/1

Query Params

Key	Value
Key	Value

Body Cookies Headers 24 Test Results

```
{  
  "albumId": 1,  
  "id": 1,  
  "title": "accusamus beatae ad facilis cum similique qui sunt",  
  "url": "https://via.placeholder.com/600/92c952",  
  "thumbnailUrl": "https://via.placeholder.com/150/92c952"  
}
```

4.

JAIRO AMIR CAHUN TAMAY's Workspace

GET https://jsonplaceholder.typicode.com/todos/1

GET https://jsonplaceholder.typicode.com/todos/1

Query Params

Key	Value
Key	Value

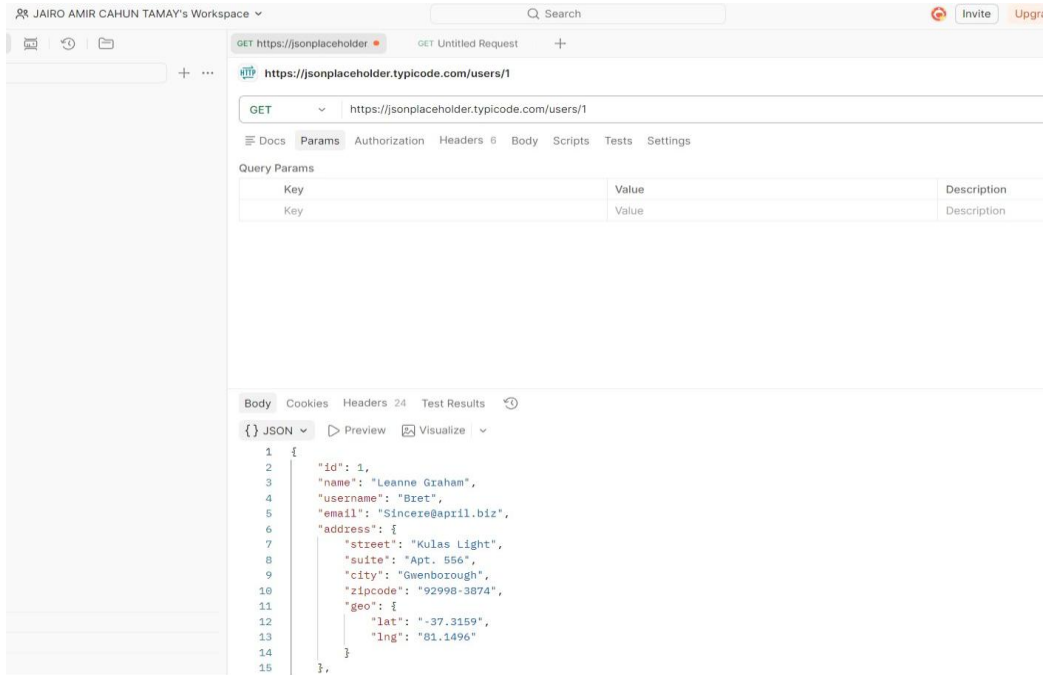
Body Cookies Headers 24 Test Results

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "delectus aut autem",  
  "completed": false  
}
```



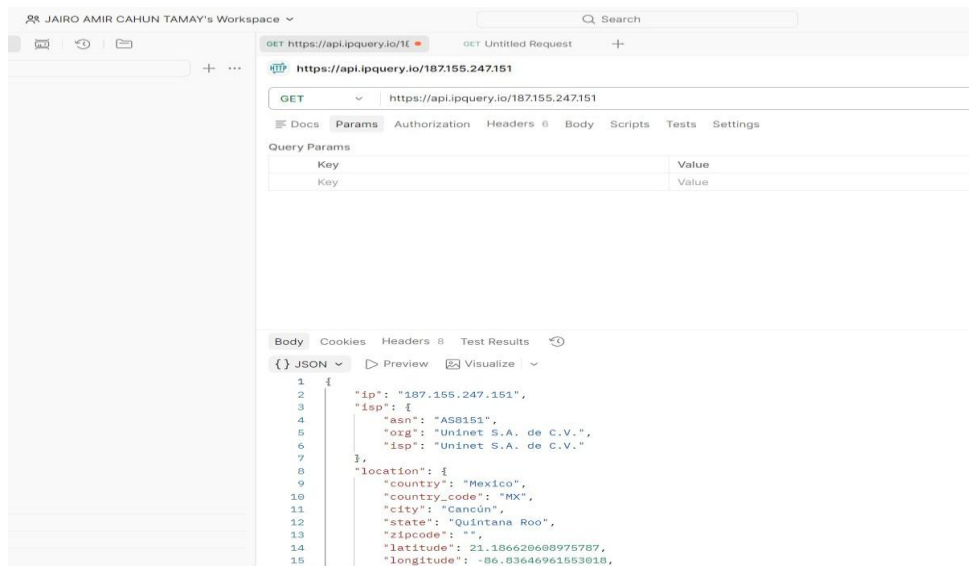


5.



Ejercicio 1

1. Extrae la información de tu IP, usando la API: [ipquery.io](https://api.ipquery.io/)
2. Accede a la documentación y realiza las pruebas con postman.
3. Crea un archivo JSON con la información obtenida.





```
13     "zipcode": "",
14     "latitude": 21.186620608975787,
15     "longitude": -86.83646961553018,
16     "timezone": "America/Cancun",
17     "localtime": "2026-03-20T04:43:01"
18 },
19 "risk": {
20     "is_mobile": false,
21     "is_vpn": false,
22     "is_tor": false,
23     "is_proxy": false,
24     "is_datacenter": false,
25     "risk_score": 0
26 }
27 }
```



2026
año de
**Margarita
Maza**





2. El formato a la peticiones API, requiere comprender los conceptos de:

- Endpoints.

URLBase/endpoint

<https://randomuser.me/api/>

<https://randomuser.me/api/1.4/>

<https://jsonplaceholder.typicode.com/comments>

- Parámetros Query.

URLBase/endpoint/query

<https://randomuser.me/api/?gender=female>

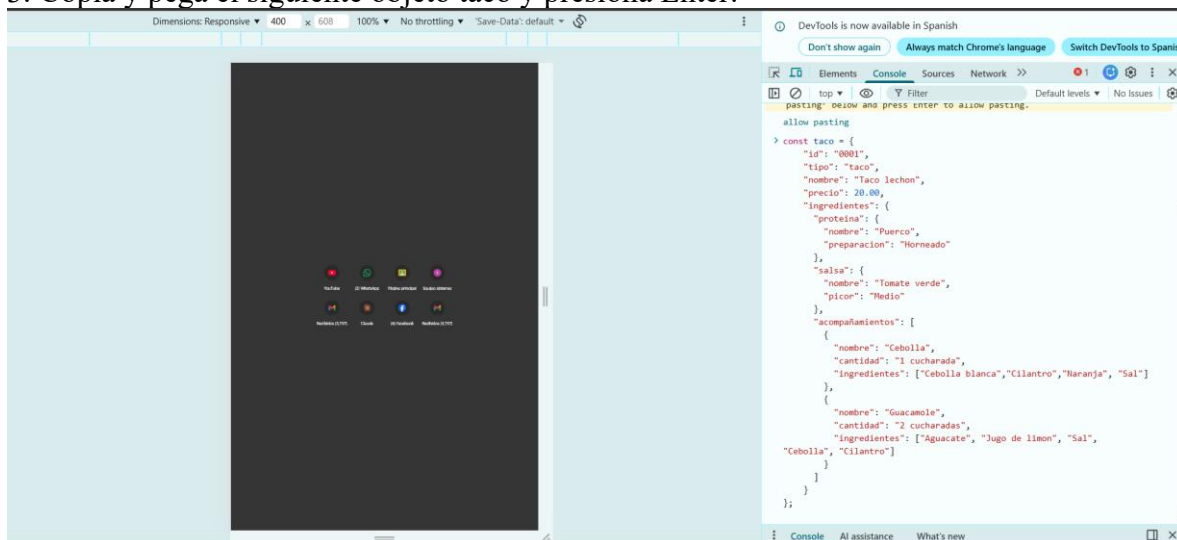
3. Manipulación de JSON y el DOM

Objetivo: Aprender a navegar por estructuras de datos complejas (objetos anidados y arreglos) utilizando la notación de punto y corchetes.

1. Abre las herramientas de desarrollador en tu navegador (F12 o Ctrl+Shift+I).

2. Ve a la pestaña Console.

3. Copia y pega el siguiente objeto taco y presiona Enter:



Escribe el comando `console.log()` necesario para extraer la siguiente información:





4. Obtén el nombre de la proteína ("Puerco").

```
> console.log(taco.ingredientes.proteina.nombre);
```

Puerco

[VM286:1](#)

5. Accede al primer elemento de la lista de ingredientes de la Cebolla ("Cebolla blanca").

```
> console.log(taco.ingredientes.acompañamientos[0].ingredientes[0]);
```

Cebolla blanca

[VM318:1](#)

6. Accede al último elemento de la lista de ingredientes del Guacamole ("Cilantro").

```
> console.log(  
  taco.ingredientes.acompañamientos[1].ingredientes[  
    taco.ingredientes.acompañamientos[1].ingredientes.length - 1  
  ]  
);
```

Cilantro

[VM322:1](#)

← undefined

Renderizar información dinámica desde un objeto JSON hacia el navegador utilizando JavaScript y manipulación del DOM.

1. Crea un archivo llamado index.html y utiliza el siguiente código base. Tu tarea es completar la logica de JavaScript para que la información del taco aparezca en la tarjeta.



2026
año de
**Margarita
Maza**





EJERCICIOS_2.1 > index.html > html > body > script

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>Receta</title>
7   <style>
8     .card {
9       border: 2px solid #2ecc71;
10      border-radius: 10px;
11      padding: 20px;
12      width: 300px;
13      font-family: sans-serif;
14    }
15
16    .precio {
17      color: #e67e22;
18      font-weight: bold;
19      font-size: 1.2em;
20    }
21
22    ul {
23      padding-left: 20px;
24    }
25  </style>
26 </head>
27
```



2026
año de
Margarita Maza





```

1 <div id="taco-card" class="card">
2   <h1 id="taco-nombre">Cargando...</h1>
3   <p class="precio">Precio: <span id="taco-precio">0</span></p>
4   <h3>Acompañamientos:</h3>
5   <ul id="lista-acompañamientos">
6   </ul>
7 </div>
8
9 <script>
10  const taco = {
11    "id": "0001",
12    "tipo": "taco",
13    "nombre": "Taco lechon",
14    "precio": 20.00,
15    "ingredientes": {
16      "proteina": {
17        "nombre": "Puerco",
18        "preparacion": "Horneado"
19      },
20      "salsa": {
21        "nombre": "Tomate verde",
22        "picor": "Medio"
23      },
24      "acompañamientos": [
25        {
26          "nombre": "Cebolla",
27          "cantidad": "1 cucharada",
28          "ingredientes": ["Cebolla blanca", "Cilantro", "Naranja", "Sal"]
29        },
30        {
31          "nombre": "Guacamole",
32          "cantidad": "2 cucharadas",
33          "ingredientes": ["Aguacate", "Jugo de limon", "Sal", "Cebolla", "Cilantro"]
34        }
35      ]
36    }
37  };
38
39 // 1. Insertar nombre en el h1
40 document.getElementById("taco-nombre").textContent = taco.nombre;
41
42 // 2. Insertar precio con formato moneda (MXN)
43 document.getElementById("taco-precio").textContent =
44   taco.precio.toLocaleString("es-MX", { style: "currency", currency: "MXN" });
45
46 // 3. Recorrer acompañamientos y crear <li>
47 const lista = document.getElementById("lista-acompañamientos");
```



2026
año de
Margarita Maza





```
74
75 // 3. Recorrer acompañamientos y crear <li>
76 const lista = document.getElementById("lista-acompañamientos");
77
78 taco.ingredientes.acompañamientos.forEach(acomp => {
79   const li = document.createElement("li");
80   li.textContent = `${acomp.nombre}: ${acomp.ingredientes.join(", ")}`;
81   lista.appendChild(li);
82 });
83
84 </script>
85 </body>
86
87 </html>
```

Receta

← ↻ ⓘ Archivo C:/Users/amirc/Downloads/tema-dos-api/EJERCICI... ☆

Taco lechon

Precio: \$20.00

Acompañamientos:

- Cebolla: Cebolla blanca, Cilantro, Naranja, Sal
- Guacamole: Aguacate, Jugo de limon, Sal, Cebolla, Cilantro



2026
año de
Margarita Maza





4. JSON (JavaScript Object Notation)

JSON es un formato ligero para el intercambio de datos, basado en la notación de objetos de JavaScript. Se utiliza ampliamente para estructurar datos que se envían a través de Internet.

Los datos en JSON se organizan en pares de clave-valor y pueden incluir arrays y objetos anidados.

La principal diferencia entre un objeto JavaScript y un JSON es que las claves en JSON deben estar entre comillas.

Objeto en JS

```
const objetoJavaScript = {  
  nombre: "Taco de Pollo",  
  ingredientes: {  
    proteina: "Pollo",  
    salsa: "Salsa Verde"  
  }  
};
```

Objeto JSON

```
{  
  "nombre": "Taco de Pollo",  
  "ingredientes": {  
    "proteina": "Pollo",  
    "salsa": "Salsa Verde"  
  }  
}
```

1. Para convertir el objeto en una cadena JSON. Crea un archivo .js en tu editor VSC y copia el código del objeto. Seguidamente copia el siguiente código:





```
EJERCICIOS_2.1 > JS serializar.js > ...
1  const objetoJavaScript = {
2      nombre: "Taco de Pollo",
3      ingredientes: {
4          proteina: "Pollo",
5          salsa: "Salsa Verde"
6      }
7  };
8
9  // Convertir a JSON
10 const jsonString = JSON.stringify(objetoJavaScript);
11
12 console.log(jsonString);
```

2. Observa el resultado.

```
amirc@amir27 MINGW64 ~/Downloads/tema-dos-api/EJERCICIOS_2.1 (master)
• $ node serializar.js
{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo","salsa":"Salsa Verde"}}
```

3. Para realizar la conversión contraria de un JSON a un objeto JS. Crea otro archivo y copia el siguiente código:

```
EJERCICIOS_2.1 > JS deserializar.js > ...
1  // Cadena JSON
2  const jsonString = '{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo","salsa":"Salsa Verde"}}';
3
4  // Convertir a objeto JS
5  const objetoDeserializado = JSON.parse(jsonString);
6
7  console.log(objetoDeserializado);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
amirc@amir27 MINGW64 ~/Downloads/tema-dos-api/EJERCICIOS_2.1 (master)
$ node deserializar.js
nombre: 'Taco de Pollo',
ingredientes: { proteina: 'Pollo', salsa: 'Salsa Verde' }
```



2026
año de
Margarita Maza





```
EJERCICIOS_2.1 > JS deserializar.js > ...
1 // Cadena JSON
2 const jsonString = '{"nombre":"Taco de Pollo", "ingredientes": {"proteina":"Pollo", "salsa":"Salsa Verde"}}';
3
4 // Convertir a objeto JS
5 const objetoDeserializado = JSON.parse(jsonString);
6
7 console.log(objetoDeserializado);
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
amirc@amir27 MINGW64 ~/Downloads/tema-dos-api/EJERCICIOS_2.1 (master)
$ node deserializar.js
  nombre: 'Taco de Pollo',
  ingredientes: { proteina: 'Pollo', salsa: 'Salsa Verde' }
}

amirc@amir27 MINGW64 ~/Downloads/tema-dos-api/EJERCICIOS_2.1 (master)
$
```

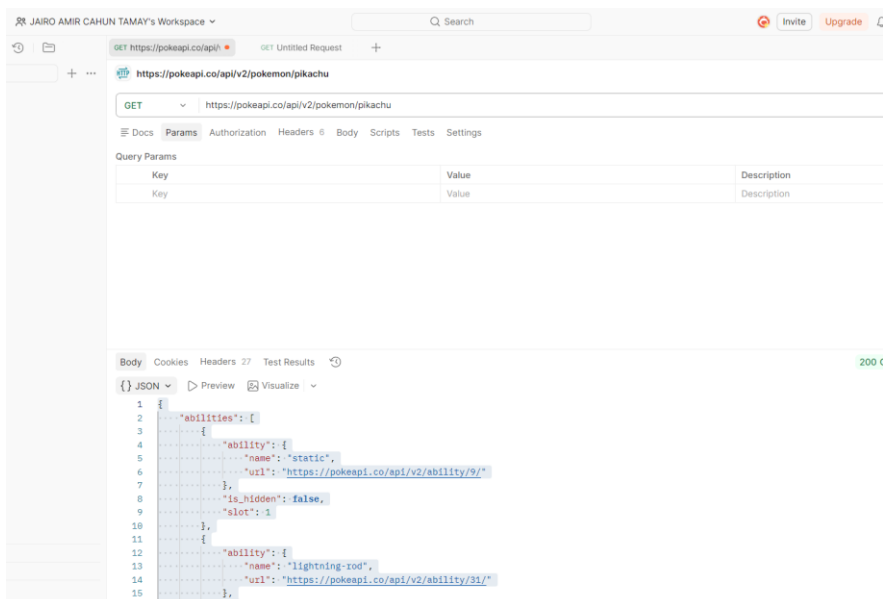
4. Ejecuta la siguiente petición GET en Postman:

<https://pokeapi.co/api/v2/pokemon/pikachu>

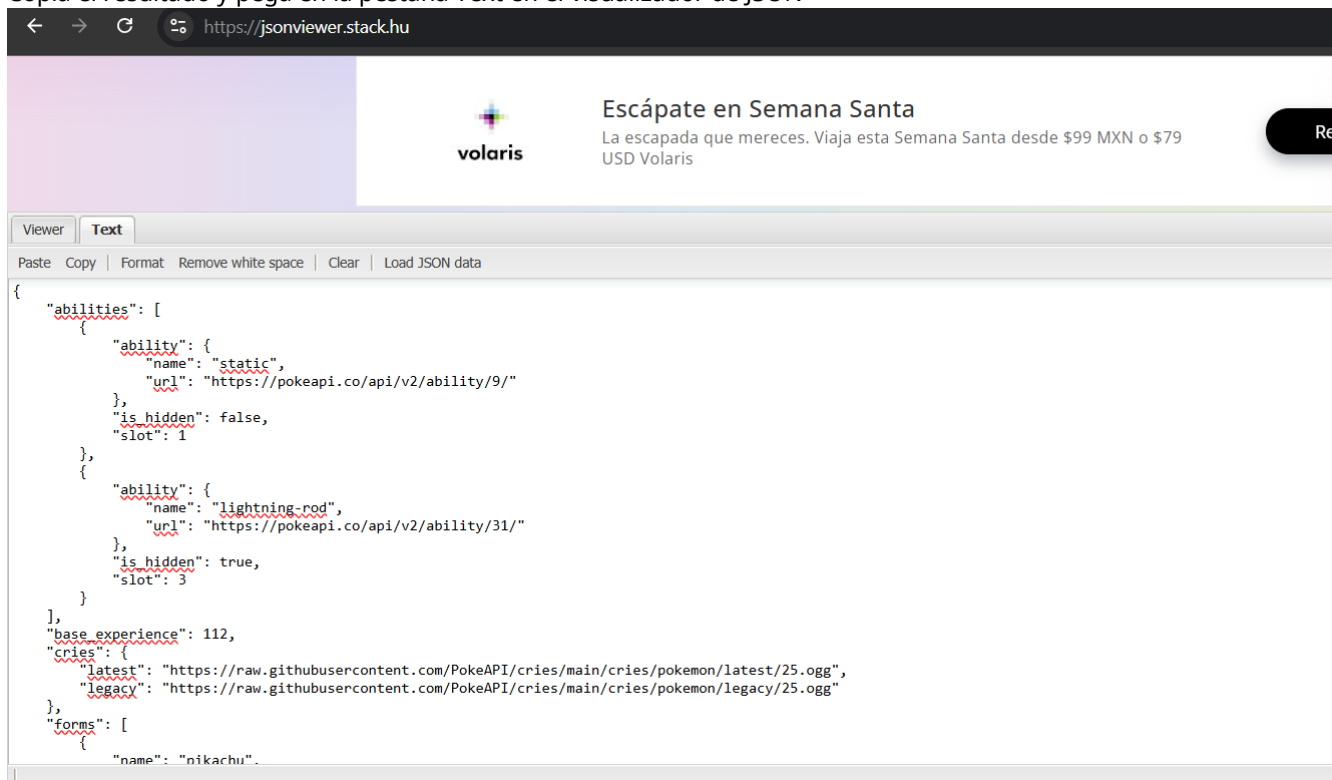


2026
año de
Margarita Maza





5. Copia el resultado y pega en la pestaña Text en el visualizador de JSON



6. Abre la pestaña Viewer e interpreta la información obtenida.





Viewer

Text

```

JSON
{
  abilities: ...
  base_experience: 112
  cries: ...
  forms: ...
  game_indices: ...
  height: 4
  held_items: ...
  id: 25
  is_default: true
  location_area_encounters: "https://pokeapi.co/api/v2/pokemon/25/encounters"
  moves: ...
  name: "pikachu"
  order: 35
  past_abilities: ...
  past_stats: ...
  past_types: ...
  species: ...
  sprites: ...
  stats: ...
  types: ...
  weight: 60

```

Name	Value
abilities	...
base_experience	112
cries	...
forms	...
game_indices	...
height	4
held_items	...
id	25
is_default	true
location_area_encounters	"https://pokeapi.co/api/v2/pokemon/25/encounters"
moves	...
name	"pikachu"
order	35
past_abilities	...
past_stats	...
past_types	...
species	...
sprites	...
stats	...
types	...

Search:

GO!

Next

Previous

♥ Support this one-nerd project, remove ads and increase loading speed all for \$19 (one-time): [Are you with me?](#)

7. Crea un archivo json con el nombre recetaTacos. Agrega 4 tacos diferentes típicos de la península, con diferente proteína y acompañamientos. Basate en el siguiente ejemplo:

```

EJERCICIOS_2.1 > {} recetaTacos.json > {} 0 > {} ingredientes > {} proteina
1  [
2    {
3      "id": "0001",
4      "tipo": "taco",
5      "nombre": "Taco lechon",
6      "precio": 20.00,
7      "ingredientes": {
8        "proteina": {
9          "nombre": "Puerco",
10         "preparacion": "Horneado"
11       },
12       "salsa": {
13         "nombre": "Tomate verde",
14         "picor": "Medio"
15       },
16       "acompañamientos": [
17         {
18           "nombre": "Cebolla",
19           "cantidad": "1 cucharada",
20           "ingredientes": [
21             "Cebolla blanca",
22             "Cilantro",
23             "Naranja",
24             "Sal"
25           ]
26         },
27         {
28           "nombre": "Guacamole",
29           "cantidad": "2 cucharadas",
30           "ingredientes": [
31             "Aguacate",
32             "Jugo de limon",
33             "Sal",
34             "Cebolla",
35             "Cilantro"
36           ]
37         }
38       ]
39     }
40   ]

```



2026
año de
Margarita Maza





```
1 {
2   "id": "0002",
3   "tipo": "taco",
4   "nombre": "Taco de cochinita pibil",
5   "precio": 18.00,
6   "ingredientes": {
7     "proteina": {
8       "nombre": "Cochinita",
9       "preparacion": "Adobado y horneado"
10    },
11    "salsa": {
12      "nombre": "Habanero",
13      "picor": "Alto"
14    },
15    "acompañamientos": [
16      {
17        "nombre": "Cebolla morada",
18        "cantidad": "1 cucharada",
19        "ingredientes": [
20          "Cebolla morada",
21          "Vinagre",
22          "Orégano",
23          "Sal"
24        ]
25      }
26    ]
27  },
28 },
29 {
30   "id": "0003",
31   "tipo": "taco",
32   "nombre": "Taco de pollo asado",
33   "precio": 15.00,
34   "ingredientes": {
35     "proteina": {
36       "nombre": "Pollo",
37       "preparacion": "Asado"
38     }
39   }
40 }
```



2026
año de
**Margarita
Maza**





```
68 },
69 {
70   "id": "0003",
71   "tipo": "taco",
72   "nombre": "Taco de pollo asado",
73   "precio": 15.00,
74   "ingredientes": {
75     "proteina": {
76       "nombre": "Pollo",
77       "preparacion": "Asado"
78     },
79     "salsa": {
80       "nombre": "Roja",
81       "picor": "Medio"
82     },
83     "acompañamientos": [
84       {
85         "nombre": "Lechuga",
86         "cantidad": "1 porcion",
87         "ingredientes": [
88           "Lechuga",
89           "Crema",
90           "Queso"
91         ]
92       }
93     ]
94   }
95 },
96 {
97   "id": "0004",
98   "tipo": "taco",
99   "nombre": "Taco de pescado",
100  "precio": 22.00,
101  "ingredientes": {
102    "proteina": {
103      "nombre": "Pescado",
104      "preparacion": "Empanizado"
```

```
amirc@amir27 MINGW64 ~/Downloads/tema-dos-api/EJERCICIOS_2.1 (master)
• $ node leerJson.js
Lista de tacos:

1. Taco lechon - $20
2. Taco de cochinita pibil - $18
3. Taco de pollo asado - $15
4. Taco de pescado - $22
```

5.



2026
año de
Margarita Maza

